Develop a contact directory using a right-threaded BST. Implement contact insertion and inorder traversal without using a stack or recursion.

1. About the Project

The Contact Directory System is a C++ project developed to manage contact names and phone numbers using a Right-Threaded Binary Search Tree (RTBST). The system allows users to insert new contacts, display all contacts in alphabetical order, and search for a contact using a menu-driven interface.

A right-threaded BST uses the right pointer of a node to point to its inorder successor when the node does not have a right child. This helps perform inorder traversal without using recursion or a stack.

2. Features

Basic Features

- Add Contact: Insert a new contact with a name and phone number.

- Display Contacts in Sorted Order: Display all contacts alphabetically using inorder traversal.

- Search Contact: Search for a contact by name.

- Duplicate Contact Checking: Prevent duplicate contact names from being inserted.

- Exit Application: Exit the program using the menu.

Contact Management Features

- Contact Identification: Use contact names as unique identifiers.

- Phone Number Storage: Store a phone number with each contact.

- Alphabetical Arrangement: Maintain contacts in sorted order using the BST.

- Threaded Traversal: Use right-thread pointers to access inorder successors.

- Memory Management: Allocate memory dynamically for each contact node.

- Future Improvement: Add contact editing and deletion functionality.

- Future Improvement: Add email addresses, home addresses, and contact categories.

3. Data Structure Used: Right-Threaded Binary Search Tree (RTBST)

Each node stores:

- Contact name.

- Phone number.

- Pointer to the left child.

- Pointer to the right child or inorder successor.

- A Boolean flag to indicate whether the right pointer is a thread.

In a right-threaded BST:

- Smaller contact names are stored in the left subtree.

- Larger contact names are stored in the right subtree.

- If a node has no right child, its right pointer points to its inorder successor.

- A right-thread flag identifies whether the right pointer represents a child or a thread.

Inorder traversal without recursion or a stack:

1. Start from the leftmost node of the tree.

2. Display the current contact.

3. If the right pointer is a thread, follow it to the inorder successor.

4. Otherwise, move to the leftmost node in the right subtree.

5. Repeat until all contacts have been displayed.

This method traverses the contacts in alphabetical order without requiring a stack or recursive function calls.

4. Technologies Used

- C++

- Structures and pointers

- Dynamic memory allocation using `new`

- Right-Threaded Binary Search Tree

- Boolean flags for thread identification

- Functions

- Iterative inorder traversal

5. Applications

- Personal contact directory management.

- Organizing contacts alphabetically.

- Searching for contact names.

- Efficient inorder traversal without recursion or a stack.

- Learning advanced tree data structures through a practical project.

6. Conclusion

The Contact Directory System demonstrates how a Right-Threaded Binary Search Tree can be used to organize and manage contact information efficiently. It supports contact insertion, duplicate checking, searching, and alphabetical display.

The use of right-threading enables inorder traversal without recursion or an auxiliary stack, making the project a practical demonstration of threaded binary trees and pointer-based data structures in C++.

7. Language

C++